

B-Tree (m way tree)

- minimum children = $m/2$
- max children = m
- minimum keys =
- maximum keys = $m-1$

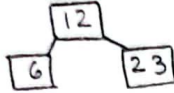
★ 5-way tree, internal nodes have children 3,4,5 and [5-1] nodes max keys.

INSERTION

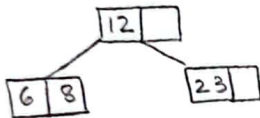
$M=3$, [12, 23, 6, 8, 15, 19, 45, 1, 4, 7, 5]

STEP 01: [12 | 23]

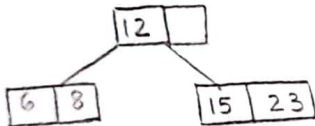
STEP 02: inserting 6, so split 12, 23



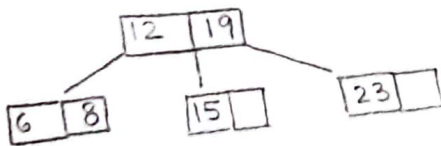
STEP 03: inserting 8



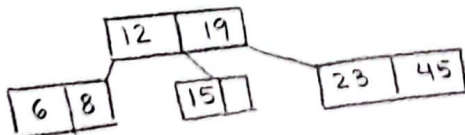
STEP 04: inserting 15



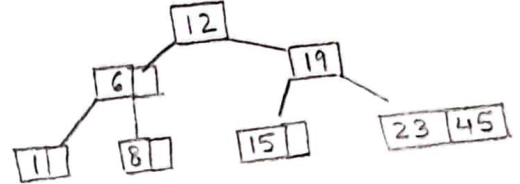
STEP 05: inserting 19, so split 15, 23



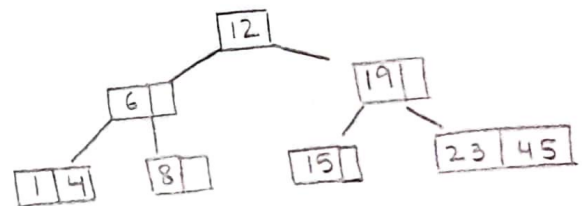
STEP 06: inserting 45



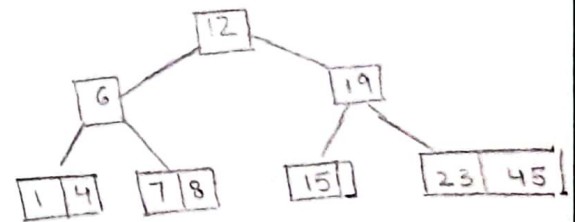
STEP 07: inserting 1, so split 6, 8 and 12, 19



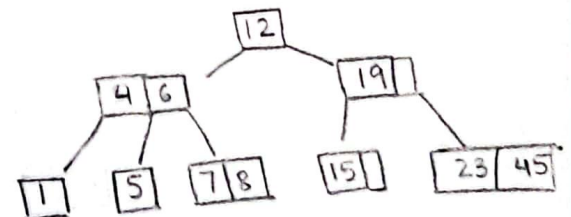
STEP 08: insert 4



STEP 09: insert 7

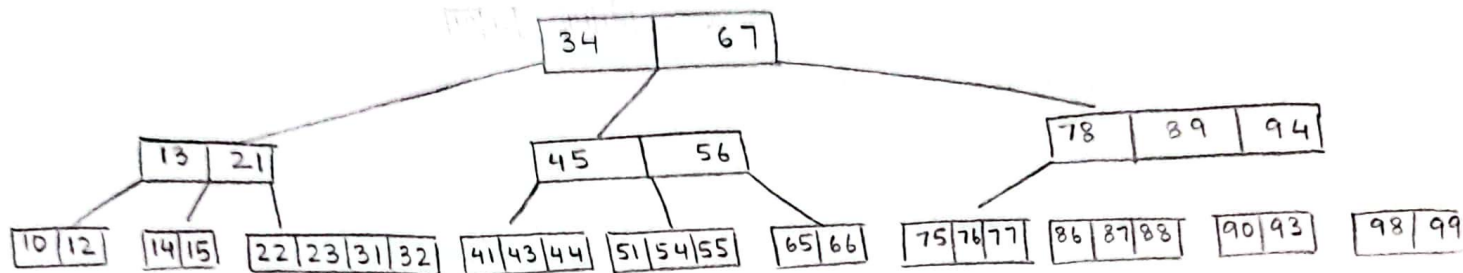


STEP 10: insert 5, so split 1, 4



DELETION

- $M = 5$
- minimum Keys = 2
- maximum Keys = 4



1. Target key in leaf node

CASE 1: leaf node has more than min Keys.

→ simply delete the key.

CASE 2: leaf node has minimum number of keys.

1. borrow maximum key from left sibling: transfer this key to the parent node, and parent node will transfer its key to the target node and replace it.

OR

2. borrow minimum key from right sibling: transfer this key to the parent node, and parent node will transfer its key to the target node and replace it.

OR

3. merge with either left or right sibling including parent key, delete target.

2. Target Key in Internal node

CASE 1: maximum element in left subtree

→ choose largest key from LST, and replace it with target key.

OR

CASE 2: minimum element in right subtree.

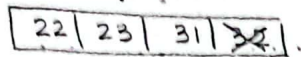
→ choose smallest key from RST, and replace it with target key.

OR

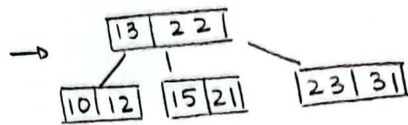
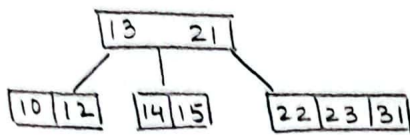
CASE 3: both case not possible

→ merge left and right nodes, along with target key, and delete it.

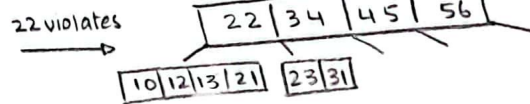
1. delete 32 → case 1 (LN)



2. delete 14 → case 2.2 (LN)

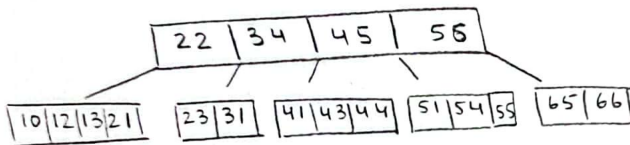


3. delete 15 → case 2.3 (LN)

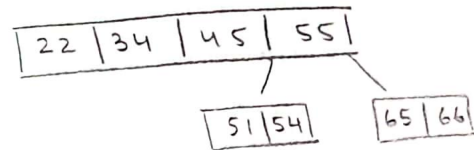


*parent of 22 is brought down

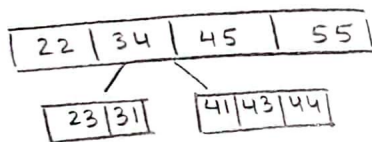
4. delete 56 → case 1 (IN)



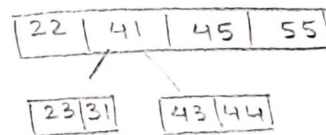
LST



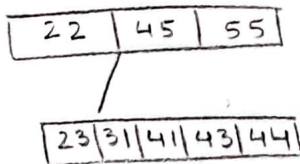
5. delete 34 — case 2 (IN)



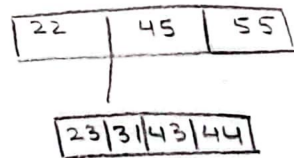
RST



6. delete 41



merge



B+ TREE

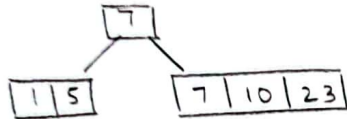
- maximum children = m
- minimum children = $m/2 = 3$
- maximum keys = $4(m-1)$
- minimum key =

INSERTION

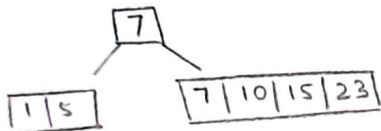
M=5, [7, 10, 1, 23, 5, 15, 17, 9, 11, 39, 35, 8, 40, 25]

STEP 01: [1 | 7 | 10 | 23]

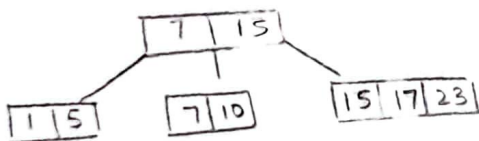
STEP 02: inserting 5



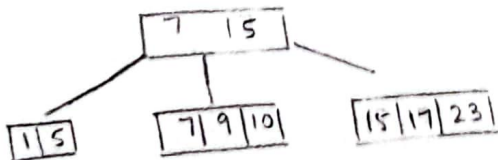
STEP 03: inserting 15



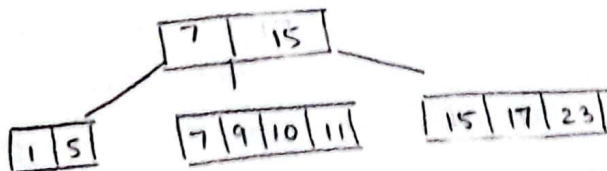
STEP 04: inserting 17



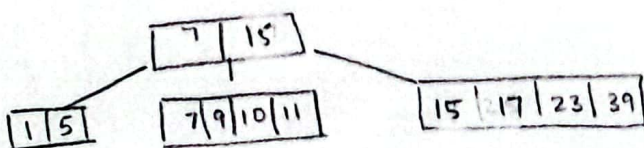
STEP 05: inserting 9



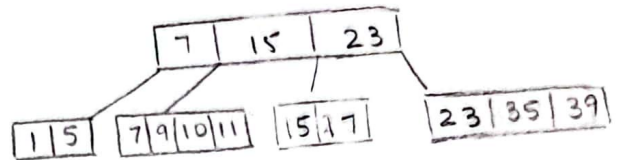
STEP 06: inserting 11



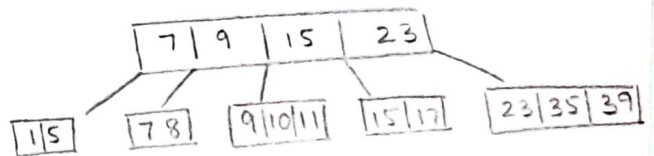
STEP 07: inserting 39



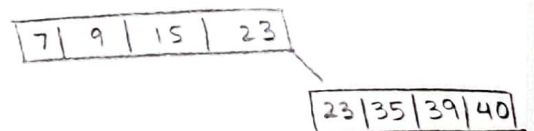
STEP 08: inserting 35



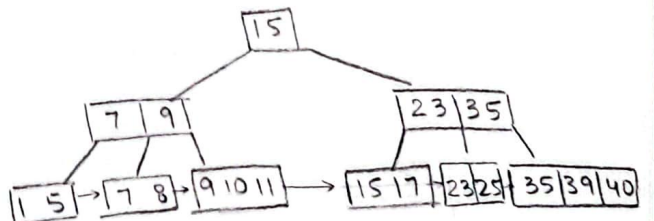
STEP 09: inserting 8



STEP 10: inserting 40



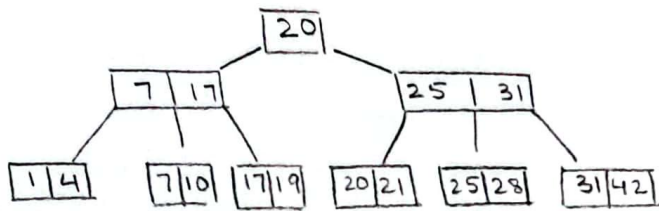
STEP 11: inserting 25



DELETION

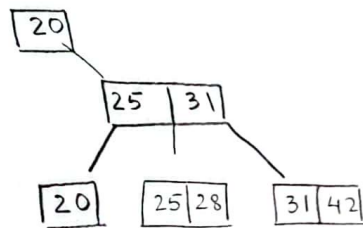
- max children = $m = 4$
- min children = $m/2 = 2$
- max keys = $m - 1 = 3$
- min keys = $\frac{m}{2} - 1 = 1$

$M = 4, [21, 30, 20, 10, 7, 25, 42]$

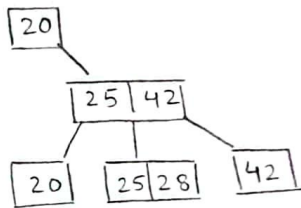


• delete 21

delete from
leaf node.

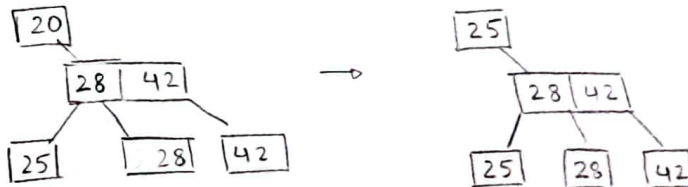


• delete 31

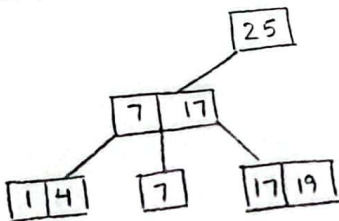


• delete 20

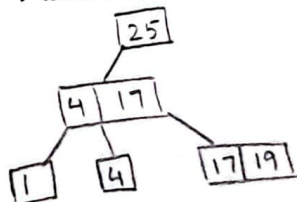
* borrow minimum
key from right
sibling.



• delete 10

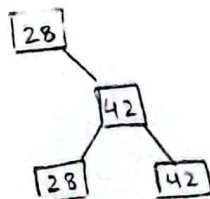


• delete 7

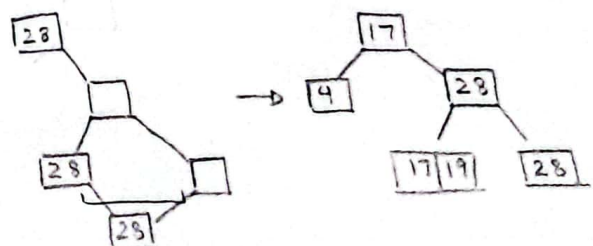


• delete 25

when merging, delete
index node between
two nodes



• delete 42



RABIN KARP ALGORITHM

Text = a a a a a b $n=6$

$a=1, b=2, c=3, d=4, e=5$

Pattern = a a b $m=3$

$$\text{pattern} = 1+1+2 = 4$$

↑
hashcode

STEP 01: a a a a a b = $1+1+1 = 3$ is not equal to 4

STEP 02: a a a a a b = $1+1+1 = 3$ is not equal to 4

STEP 03: a a a a a b = $1+1+1 = 3$ is not equal to 4

STEP 04: a a a a a b = $1+1+2 = 4$ is equal to 4, pattern found.

Text = a b c d a b c e $n=8$

Pattern = b c e $m=3$

$$\text{Pattern} = 2+3+5 = 10$$

STEP 01: a b c d a b c e = $1+2+3 = 6$ is not equal to 10

STEP 02: a b c d a b c e = $2+3+4 = 9$ is not equal to 10

STEP 03: a b c d a b c e = $3+4+1 = 8$ is not equal to 10

STEP 04: a b c d a b c e = $4+1+2 = 7$ is not equal to 10

STEP 05: a b c d a b c e = $1+2+3 = 6$ is not equal to 10

STEP 06: a b c d a b c e = $2+3+5 = 10$ is equal to 10, pattern found

- spurious hit when hash number matches the pattern number, but is actually not.
- worst case $\rightarrow O(mn)$
- no spurious hits $\rightarrow O(n-m+1)$
- improve hash function

Text: e c a c c a a e d b a

$$\text{Pattern: dba} = 4 \times 10^2 + 2 \times 10^1 + 1 \times 10^0 = 421$$

KNUTH-MORRIS PRATT ALGORITHM

pattern 1 : a¹b²c³d⁴ a⁵b⁶ e⁷ a⁸b⁹ f¹⁰

π : 0 0 0 0 1 2 0 1 2 0

pattern 2 : abcde abf abc

π : 0 0 0 0 0 1 2 0 1 2 3

pattern 3 : aabcad aabe

π : 0 1 0 0 1 0 1 2 0 0

pattern 4 : aaaabaacd

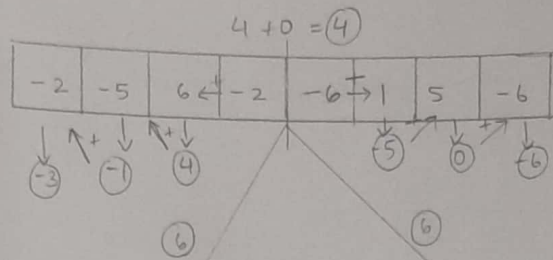
π : 0 1 2 3 0 1 2 0 0

string : a b a b c a b c a b a b a b d

	0	1	2	3	4	5
pattern	a	b	a	b	d	
π	0	0	1	2	0	

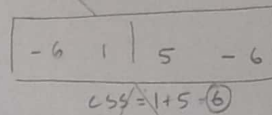
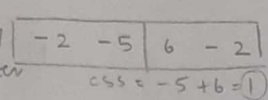
- start with $j=0$ in pattern and $i=1$ in string.
- match $j+1$ with i , if a match is found, increment both i and j
- if a match is not found, then backtrack only j to the index of which it is pointing right now.
- do this until end of string.
- $O(n+7m)$ where n = searching, m = π table backtracking.

MAXIMUM SUB ARRAY



6 - 2 = 4
6 is greater

-5 - 2 = -7
-5 is greater



5 - 6 = -1
5 is greater

-6 + 1 = -5
1 is greater

LSS = -2

RSS = 6

LSS = 1

RSS = 5

* maximum
from CSS, LSS,
RSS

-2 - 5

6 - 2

-6 1

5 - 6

CSS = -2 - 5 = -7

CSS = 6 - 2 = 4

CSS = -6 + 1 = -5

CSS = 5 - 6 = -1

LSS = -2

RSS = -5

LSS = 6

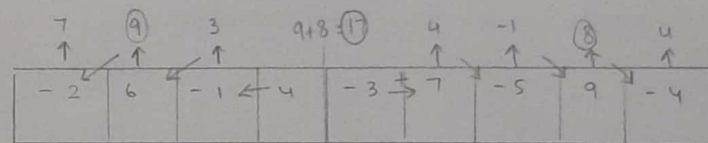
RSS = -2

LSS = -6

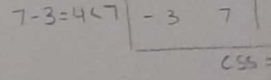
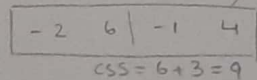
RSS = 1

LSS = 5

RSS = -6



-1 + 4 = 3 > 1
6 - 2 = 4 < 6



-5 + 9 = 4
4 - 4 = 0

LSS = 6

RSS = 4

LSS = 7

RSS = 9

-2 6

-1 4

-3 7

-5 9

CSS = 4

CSS = 3

CSS = 4

CSS = 9 - 5 = 4

LSS = -2

RSS = 6

LSS = -1

RSS = 4

LSS = -3

RSS = 7

LSS = -5

RSS = 9

-2

6

-1

4

-3

7

-5

9

-4

CSS = 5

LSS = 9

RSS = 4

9

-4

- divide and conquer - $O(n \log n)$
- kadane's algorithm - $O(n)$
- Brute force - $O(n^3)$

BUCKET SORT

- works on floating point numbers between range 0.0 to 1.00

0.79, 0.13, 0.64, 0.39, 0.20, 0.89, 0.53, 0.42, 0.06, 0.94

0	→ 0.06
1	→ 0.13
2	→ 0.20
3	→ 0.39
4	→ 0.42
5	→ 0.53
6	→ 0.64
7	→ 0.79
8	→ 0.89
9	→ 0.94

- the number after decimal is where we need to place our number in bucket.
- multiplying the number with 10 will result in 0 be removed.
- now consider lower value of number and place on that bucket.
- best case $O(n)$

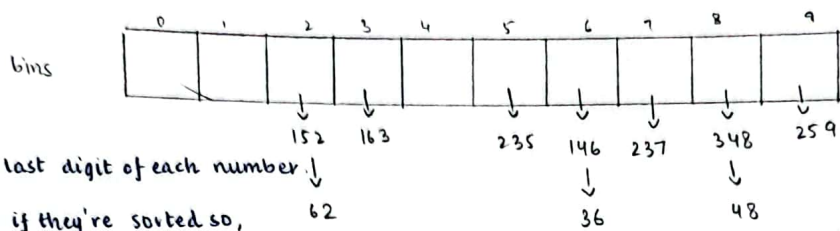
worst case → 0.79, 0.78, 0.795, 0.74

7	→ 0.79, 0.78, 0.795, 0.74

- apply insertion sort on individual buckets and sort it on incoming values.
- $O(n^2)$

RADIX SORT

237	146	259	348	152	163	235	48	36	62
0	1	2	3	4	5	6	7	8	9



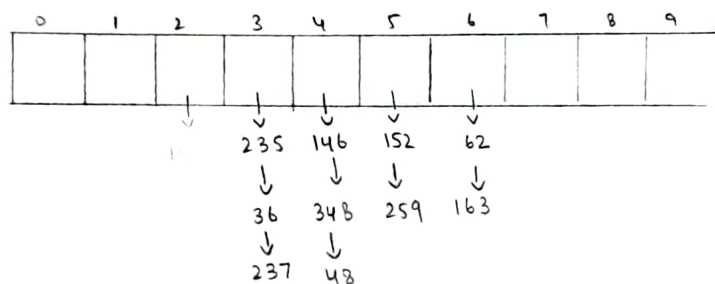
• check last digit of each number ↓

• check if they're sorted so,

152, 62, 163, 235, 146, 36, 237, 348, 48, 259

• if not sorted, then drop second last digit from right in the bin again.

$$\rightarrow (A[i] / 10) \% 10$$

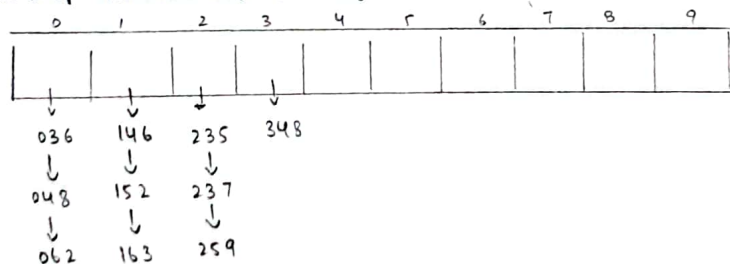


• check if they're sorted,

235, 36, 237, 146, 348, 48, 152, 259, 62, 163

$$\rightarrow (A[i] / 100) \% 10$$

• not sorted, then drop third last digit from right.



• check if sorted.

36, 48, 62, 146, 152, 163, 235, 237, 259, 348

• $O(n) = O(dn)$

COUNTING SORT

- sorting according to keys
- counting elements having distinct key values.

array: 1, 0, 2, 1, 0, 1, 1, 5, 6, 7, 5, 4, 2, 2, 0, 0, 1

number of elements (n) = 17

range = 0 - 7

K = 7

array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
2	1	1	0	2	5	4	0	2	8	7	7	9	2	0	1	9

n = 17, range = 0 - 9, K = 9

count:

0	1	2	3	4	5	6	7	8	9
3	3	4	0	1	1	0	2	1	2

* initialize it with 0

* make array of K + 1 to store count.

* increment i in array and also increment that index in count array

• update count array to find actual position.

0	1	2	3	4	5	6	7	8	9
3	6	10	10	11	12	12	14	15	17
0	3	7	12						

final array:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
0	0	0	1	1	1	2	2	2	2	4	5	7	7	8	9	9

• scan the original array from right to left

• go to that index in the count array, decrement by 1 and store the original value in that count array index in final array

$$O(n+k) = O(n)$$

• k should be feasible.